

CprE 4870/5870

Lab 5: Hardware Integration and Evaluation

“The answer may be a blend, but, as of now, we don’t understand how to blend the ideas from these approaches together. [...] We need to think about hardware and software as a continuum, not as separate parts.” – **John L. Hennessy**

1 Learning Objectives

By the end of this lab you should be able to

- Debug interactions between software and hardware quickly and efficiently
- Perform an end to end inference of each of your quantized neural network implementations, with the arithmetic of all tensor operations being performed in hardware.
- Understand, demonstrate, and report power performance and area for each of your hardware designs, and intelligently analyze the tradeoffs made
- Explore and evaluate the tradeoffs between different MAC unit designs
- Design and implement a variable-precision MAC unit that is able to change quantization level at runtime. With hardware reuse, i.e. not just muxing between an 8 bit, 4 bit, and 2 bit multiplier.

2 Pre-lab

2.1 Lab Checklist

- Complete labs 3 and 4, as you will be combining your implementations from these labs to complete lab 5.
- Be able to use the ILA to identify malformed AXI Stream transactions, and view any issues that may still be present in your hardware.
- Be able to robustly debug any issues in your system. Have helper functions and behavior, either in C++, python, or your language/method of choice that let you visualize your outputs, identify malformed behaviors, and correct them in a timely manner (DO NOT just see that you have bad cosine similarity and spend 8 hours staring at your code. Identify the issue, narrow down where it’s located, and then attempt to fix it).

2.2 Intro

Now that you have implemented your neural network in a manageable, quantized precision, as well as previously implemented hardware that can manage that level of quantization, it is time to integrate both sides together and evaluate your implementation. You will also be attempting to

construct a variable precision MAC unit so that you might mix and match layer quantizations at runtime to balance tradeoffs.

This lab should realistically take your group **one week** to complete.

3 Integration Bliss

Using the functions you developed for the software testing portion of lab 3, you will create a new *computeAccelerated* function within your framework that implements quantized inference for your neural network with all of the multiply-accumulate operations being performed on your MAC unit.

1. A few changes will need to be made to your repository. A new layer inference type will need to be added in the switch statement near `Model.cpp:28`, in the enum of `InfType` near `layers/Layer.h:102`, and new `computeAccelerated` functions in both the `Layer` class near `layers/Layer.h:128` and all of your layer subclasses.
2. Only convolution and dense will need new implementations. Since maxpool and flatten don't require arithmetic, you can simply call *computeNaive* inside the function body for *computeAccelerated*. As for softmax, since it takes up such a small portion of your runtime, and requires precise arithmetic with results between 0 and 1, you can keep it implemented as a 32 bit floating point operation.
3. Implement 4-bit and 2-bit versions of both your staged and pipelined MAC. This will result in 6 MACs total if you include your 8-bit implementations from lab 3. During the process of creating an XSA for each MAC, it is recommended that you refer to the section **The Big 3 - Power, Performance, and Area** and record all necessary information for the completion of this section that can be found in your synthesis/implementation reports. Or better yet, save the reports so you can refer to them later. You will also be required to save each XSA for submission.
4. Implement the *computeAccelerated* function to utilize your MAC units, and ensure that you can perform a full inference of your neural network on all of them. This inference should match your quantized implementations directly, since the only difference is that you are performing arithmetic on your MAC unit instead of the ARM core

4 The Big 3 - Power, Performance, and Area (PPA)

Now that we have designed six flavors of a multiply accumulate unit and the ability to use them to inference our neural network we can use Vivado and some software profiling to help us analyze and compare them. Complete the below for all the MAC units you have designed.

1. Based off of your design implementation, report the maximum throughput and minimum latency of your design in the form of an equation and example values. Your equation should be in terms of the number of multiply accumulates done and you can assume that there is no external back-pressure, control, or data-movement delays. Please provide some reasonable example latencies and throughputs (1 MAC, 5 MACs, 100 MACs, etc.).
2. Report the peak power and the area of your design from Vivado's built-in tools (e.g., the estimated power and energy report and utilization report). Calculate the energy required to run a single inference on our Toy Model (See Lab 1). You may assume that control and data

movement overheads are negligible and that your MAC unit will be operating at optimal throughput (it does not need to wait on any external factors). Show your work/calculations.

3. How would the power, energy, area, and performance (throughput & latency) scale if you used more than a single Processing Element (PE) / MAC unit (3 PEs, 5 PEs, 100 PEs, etc.)?
4. Do these values make sense? Do they over estimate or underestimate the realistic values we would expect to see on the board? What other factors are we not taking into account that could affect the previously calculated values?
5. Profile the execution time of all of your implementations. Compare the speed you got for each and the reasons for their differences.
6. Given the differences between the quantizations, how might a variable-precision MAC unit that allows different quantization levels on each layer affect these metrics?

5 Variable-precision MAC

Previously, a full pass of your neural network was limited to one quantization level at runtime, but what if we could change what precision our MAC uses at runtime to leverage advantages of different quantization levels for different layers? This is what a variable MAC enables, beyond simply multiplexing between 3 different precision multipliers, a variable MAC lets us utilize the same multiplier hardware for different precisions.

1. You will implement your choice of variable precision MAC unit from your textbook (EPDNN 7.4 Page 160), this includes temporal accumulation, spatial accumulation, and bit-serial. Please discuss with your TA to ensure that not everyone chooses the same method – this will make a more interesting demo.
2. Now we will use your variable-precision MAC to estimate what sort of benefit (if any) your neural network will gain from such a hardware feature. To do this you will first need to estimate what an appropriate quantization level would be for each layer and then implement a MAC that can support each level of quantization.
3. Determine a quantization level for each layer. Report your level of quantization selected and how/why you chose the levels. Note that this is a fairly open-ended question. I would suggest you start by considering quantization error and possibly perform a couple of different end-to-end accuracy evaluations to estimate the final accuracy.
4. Implement this design. Since different layers may require different quantization bit-widths at runtime, you will have to determine how to solve that by updating your MAC unit from software, as your hardware design cannot change at runtime.
5. Evaluate and report the estimated performance, area, power, and energy benefits of your quantization. Compare with the static results from above for 8, 4, and 2-bit quantization of the entire model. What was the main driver and/or limiter of each metric's improvement?

6 Demo

As part of our class demo, I would like to put together a collaborative design-space evaluation. The intent is to students choose to implement different variable-precision MACs, different static MACs,

and report the end-to-end accuracy, performance, power, and energy #s the day before the demo session so we can plot them and discuss them during the demo session. **Please update the shared spreadsheet by the evening before the demo.**

7 Above & Beyond 🚀

- Perform and evaluate a quantization-aware training (bounty points and/or project)
- Perform and evaluate a neural architecture search (NAS) for variable or reduced-precision architectures.
- Employ and evaluate a different quantization method besides linear – how does it impact accuracy?
- Implement and evaluate the zeroing method provided in the TensorFlow Lite documentation. https://www.tensorflow.org/lite/performance/quantization_spec

8 Submission Instructions 📁

Below is a list of the required turn-in components for the lab to be submitted to Canvas. Only the final submission will be graded, although you are free to resubmit as many times as you please. All submissions must be complete. Even slight changes require a complete re-submission, **NOT** a partial, single file, change. Naming conventions for each file are also provided below. <Lab Partners Number> represents the number in brackets from your Canvas group's name.

8.1 Turn-In Checklist

- PDF of your lab document, either from the template or your own format containing all the required lab components (`lab5_report_<Lab Partners Number>.pdf`).
- Completed software integrations of all of your MACs, along with the hardware they run on. A TA should be able to use your software and hardware XSA, and simply run the create and flash vitis scripts to ensure your implementations work.

8.2 File Structure

If you followed the directions from previous labs and made a single configurable framework, simply copy this framework 4 times in the submission and configure it for inference at each bit level in each folder, that way TAs don't have to reverse engineer your code. If this is infeasible, provide clear directions on how to change quantization level

Each Canvas submission:

```
├─ lab5_report_<Lab Partners Number>.pdf
├─ lab5_src_<Lab Partners Number>.tgz/zip
│   └─ util
│       ├── lab5_<Lab Partners Number>.ipynb
│       └─ <any helper scripts you may have used>
└─ sw
    └─ 8bit_integrated_framework
        └─ ...
```

